Taxi Service Management System: Database Requirements and Queries

Tirth Kansagra 202303055
Rom Tala 202303051
Virat Srimali 202303061

August 6, 2024

1 Problem Description

The Taxi Service Management System is designed to manage the operations of a taxi service. It includes user registration, ride booking, driver management, vehicle management, feedback collection, and manager responsibilities. The system needs to handle customer and driver information, ride details, and vehicle allocation efficiently. The goal is to provide a seamless experience for both customers and drivers while ensuring robust data management and reporting capabilities.

2 Database Requirements

2.1 Entities and Attributes

1. **Sign Up Entity**
   - Signup ID: Unique identifier for each signup request.
   - Username: Chosen username for the new user.
   - Password: Securely hashed password provided during signup.
   - Name: Full name of the new user.
   - Phone Number: Contact number of the new user.
   - Email: Email address of the new user.
   - Role: Indicates whether the user is registering as a customer or a driver.
   - Status: Status of the signup request (Pending, Verified, Rejected).
2. **Login Entity**
   - Login ID: Unique identifier for each login record.
   - Username: Login username for the user.
   - Password: Securely hashed password for authentication.
3. **Customer Entity**
   - Customer ID: Unique identifier for each customer.
   - Name: Full name of the customer.
   - Phone Number: Contact number of the customer.
   - Email: Email address of the customer.
4. **Driver Entity**
   - Driver ID: Unique identifier for each driver.
   - Name: Full name of the driver.
   - Phone Number: Contact number of the driver.

- Email: Email address of the driver.
- Vehicle Details: Information about the vehicle, such as make, model, and license plate.
- Availability Status: Current availability status of the driver.

5. **Ride Booking Entity**
   - Ride ID: Unique identifier for each ride.
   - Customer ID: Identifier for the customer who requested the ride.
   - Driver ID: Identifier for the driver assigned to the ride.
   - Pickup Location: Address or coordinates of the pickup location.
   - Drop-off Location: Address or coordinates of the drop-off location.
   - Fare: Calculated fare for the ride.
   - Status: Current status of the ride (Pending, Completed, Cancelled).
6. **Feedback Entity**
   - Feedback ID: Unique identifier for each feedback entry.
   - Customer ID: Identifier for the customer providing the feedback.
   - Driver ID: Identifier for the driver receiving the feedback.
   - Rating: Rating given by the customer (e.g., 1 to 5 stars).
   - Comments: Additional comments or feedback provided by the customer.
7. **Location Entity**
   - Address ID: Unique identifier for each address.
   - Street: Street name of the address.
   - City: City name of the address.
   - State: State name of the address.
   - Pin Code: pin code of the address.
8. **Taxi Entity**
   - Taxi ID: Unique identifier for each vehicle.
   - Driver ID: Identifier for the driver using the vehicle.
   - Make: Manufacturer of the vehicle.
   - Model: Model of the vehicle.
   - License Plate: License plate number of the vehicle.
9. **Driver Shift Entity**
   - Shift ID: Unique identifier for each driver shift.
   - Driver ID: Identifier for the driver assigned to the shift.
   - Shift Start Time: Start time of the shift.
   - Shift End Time: End time of the shift.
10. **Administrator Entity**
    - Administrator ID: Unique identifier for each Administrator.
    - Name: Full name of the Administrator.
    - Phone Number: Contact number of the Administrator.
    - Email: Email address of the Administrator.

2.2 Relationships

1. Each login record must be associated with a single sign-up record, ensuring a one-to-one relationship between Login and Sign Up entities.
2. Each customer can request multiple rides, establishing a one-to-many relationship between Customer and Ride entities.
3. Each driver can handle multiple rides, establishing a one-to-many relationship between Driver and Ride entities.
4. Each customer can provide feedback for multiple rides, establishing a one-to-many relationship between Customer and Feedback entities.
5. Each driver can receive feedback from multiple customers, establishing a one-to-many relationship between Driver and Feedback entities.
6. Each ride must have a pickup location, establishing a one-to-one relationship between Ride and Address (Pickup Location) entities.
7. Each ride must have a drop-off location, establishing a one-to-one relationship between Ride and Address (Drop-off Location) entities.
8. Each driver is assigned one vehicle, but a vehicle can be reassigned to different drivers over time, establishing a one-to-many relationship between Driver and Vehicle entities.
9. Each driver can have multiple shifts, establishing a one-to-many relationship between Driver and Driver Shift entities.
10. Each ride can be allocated by a manager, establishing a many-to-one relationship between Ride and Manager entities.
11. Each manager can allocate multiple vehicles to drivers, establishing a one-to-many relationship between Manager and Driver entities.
12. Each manager can allocate multiple drivers to customers, establishing a one-to-many relationship between Manager and Ride entities.

3 Relationships and Requirements

We have expanded the relationships to meet the requirement of 20 total relationships.

3.1 Binary Relationship

1. **Customer-Ride Booking (1:N)**
   - **Requirement 1**: Each customer can make multiple bookings, but each booking must be associated with one and only one customer.
   - **Cardinality**: One-to-Many
   - **Participation**: Partial for Customer, Total for Ride Booking
2. **Driver-Ride Booking (1:N)**
   - **Requirement 2**: Each driver can handle multiple bookings, but each booking must be associated with one and only one driver.
   - **Cardinality**: One-to-Many
   - **Participation**: Partial for Driver, Total for Ride Booking
3. **Taxi-Booking (1:N)**
   - **Requirement 3**: Each taxi can be used for multiple bookings, but each booking must involve one and only one taxi.

- **Cardinality**: One-to-Many
- **Participation**: Partial for Taxi, Total for Booking

4. **Booking-Payment (1:1)**
   - **Requirement 4**: Each booking must have one and only one associated payment, and each payment must correspond to one and only one booking.
   - **Cardinality**: One-to-One
   - **Participation**: Total for both Booking and Payment
5. **Booking-Ride Feedback (1:1)**
   - **Requirement 5**: Each booking may have one and only one ride feedback, and each feedback corresponds to one booking.
   - **Cardinality**: One-to-One
   - **Participation**: Partial for both Booking and Ride Feedback
6. **Customer-Ride Feedback (1:N)**
   - **Requirement 6**: Each customer can give multiple feedbacks for different bookings, but each feedback must be associated with one and only one customer.
   - **Cardinality**: One-to-Many
   - **Participation**: Partial for Customer, Total for Ride Feedback
7. **Driver Schedule (1:N)**
   - **Requirement 7**: Each driver can have multiple schedules, but each schedule must belong to one and only one driver.
   - **Cardinality**: One-to-Many
   - **Participation**: Partial for Driver, Total for Driver Schedule
8. **Location-Booking (M:N)**
   - **Requirement 8**: Each booking has a pickup and dropoff location, and each location can be associated with multiple bookings.
   - **Cardinality**: Many-to-Many
   - **Participation**: Partial for Location, Total for Booking
9. **Administrator-Driver (1:N)**
   - **Requirement 9**: Each administrator can manage multiple drivers, but each driver must be managed by one and only one administrator.
   - **Cardinality**: One-to-Many
   - **Participation**: Partial for Administrator, Total for Driver
10. **Driver-Taxi (1:1)**
    - **Requirement 10**: A driver may be assigned to one and only one taxi at a time, and a taxi may be driven by one and only one driver at a time.
    - **Cardinality**: One-to-One
    - **Participation**: Partial for both Driver and Taxi
11. **Customer-Referral (1:N)**
    - **Requirement 11**: A customer can refer multiple other customers, forming a recursive relationship where a customer can be both a referrer and a referee.
    - **Cardinality**: One-to-Many
    - **Participation**: Partial for both Referring Customer and Referred Customer

12. **Customer-Payment (1:N)**
    - **Requirement 12**: Each customer can have multiple payment records, but each payment must be associated with one and only one customer.
    - **Cardinality**: One-to-Many
    - **Participation**: Partial for Customer, Total for Payment
13. **Location-DriverSchedule (1:N)**
    - **Requirement 13**: Each driver schedule can be associated with one location (e.g., starting point), and each location can have multiple driver schedules.
    - **Cardinality**: One-to-Many
    - **Participation**: Partial for both Location and DriverSchedule
14. **Ride Booking-Payment (1:1)**
    - **Requirement 14**: Each ride booking must have one and only one payment record.
    - **Cardinality**: One-to-One
    - **Participation**: Total for both Ride Booking and Payment
15. **Promo code-Ride Booking (1:N)**
    - **Requirement 15**: Each promotion code can be applied to multiple ride bookings, but each booking must have at most one promotion code applied.
    - **Cardinality**: One-to-Many
    - **Participation**: Partial for Promotion, Total for Ride Booking
16. **Driver-License (1:1)**
    - **Requirement 16**: Each driver must have one and only one license record.
    - **Cardinality**: One-to-One
    - **Participation**: Total for both Driver and License

3.2 Ternary Relationships

1. Customer - Taxi - Booking

- **Requirement 19**: A customer can book a taxi, involving a specific taxi and creating a booking record. Each booking is associated with one customer, one taxi, and one unique booking record.
- **Cardinality:** Many-to-Many-to-Many
- **Participation:** Total for Booking, Partial for Customer and Taxi
- **Details:** This relationship encapsulates the process where customers interact with taxis through bookings, with the system recording the customer, taxi, and booking details.

2. Booking - Driver - Payment

- **Requirement:** A booking involves a driver and results in a payment transaction. Each payment is associated with one booking and one driver.
- **Cardinality:** Many-to-Many-to-One
- **Participation:** Total for Booking and Payment, Partial for Driver
- **Details:** Payments are made for services rendered by drivers, with each payment linked to a specific booking and driver.

3.3 Recursive Relationships

1. Customer - Referrals (Refer-a-Friend)

- **Requirement:** A customer can refer other customers, creating a recursive relationship where a customer can be both a referrer and a referee.
- **Cardinality:** One-to-Many
- **Participation:** Partial for both ReferringCustomer and ReferredCustomer
- **Details:** This relationship tracks the referrals made by customers, with each customer potentially referring multiple new customers.

2. Driver - Mentorship

- **Requirement:** Experienced drivers can mentor newer drivers, creating a hierarchical relationship between mentor and mentee drivers.
- **Cardinality:** One-to-Many
- **Participation:** Partial for both Mentor and Mentee
- **Details:** This relationship supports the professional development of drivers, with experienced drivers providing guidance to less experienced colleagues.

Conclusion

These relationships encapsulate the various interactions and dependencies between entities within the Taxi Management System, providing a comprehensive and detailed database structure.

Weak Entities

In the context of the Taxi Management System, weak entities are identified as entities that do not have a primary key of their own and depend on a "strong" or "owner" entity for unique identification. The following weak entities are defined:

1. RideFeedback

- **Owner Entity:** Ride Booking
- **Reason:** RideFeedback is dependent on the Booking entity because a feedback record cannot exist without a corresponding booking. The FeedbackID alone cannot uniquely identify feedback across the system without the BookingID.

2. DriverSchedule

- **Owner Entity:** Driver
- **Reason:** DriverSchedule is dependent on the Driver entity. A schedule record must be associated with a driver, and the combination of DriverID and ScheduleID uniquely identifies a schedule.

3. BookingLocation

- **Owner Entity:** Booking
- **Reason:** BookingLocation depends on the Booking entity. The locations (pickup and dropoff) for a specific booking are identified by the combination of BookingID and LocationID.

Conclusion

These weak entities are inherently dependent on their corresponding owner entities and cannot exist independently. They are typically used to represent detailed information or relationships that are specific to their owner entities.

4 Questions/Queries

4.1 Ride Statistics

1. How many rides have been completed in the last month?
2. How many rides were requested from a specific pickup location?
3. How many rides were cancelled in the last week?
4. What is the average fare for rides completed in the last month?
5. How many rides were completed by each driver during their shifts?
6. How many rides have been allocated by a specific manager?
7. What are the most common drop-off locations for rides?

4.2 Customer Statistics

1. How many new customers signed up in the last week?
2. How many customers provided feedback in the last month?
3. What are the top 5 pickup locations based on ride requests?

4.3 Driver Statistics

1. Which driver has completed the most rides?
2. What is the average rating for each driver?
3. How many shifts has a particular driver completed in a given period?
4. How many rides did each driver complete in a specific period?
5. What is the total number of active drivers?

4.4 Feedback Statistics

1. What is the distribution of ratings given by customers?
2. What is the average rating given by customers to drivers?

4.5 Sign-Up and Login Statistics

1. What is the total number of sign-up requests still pending verification?
2. How many sign-up requests were verified in the last month?
3. What is the total number of logins in the last week?

4.6 Fare and Revenue Statistics

1. What is the total fare collected for rides in a given period?
2. What is the average fare per ride over the last month?
3. What is the total revenue generated by each driver in a specific period?